claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a2505a4c457f83eac.jsonl`
- SHA-256: `0ca63ea814fed0dca1fabff480ef9e81f2df9ee0d1a3742a30f4aa91412fdec2`
- Source modified: `2026-06-10T06:14:29+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T06:12:46.618Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
ROLE: adversarial verifier. An auditor ("eng-pipeline") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.
FINDING: {"title":"stitching.begin_padding/end_padding config is not wired into the live compile paths ? API reels always get end_padding=1.0 (config says 0.5)","severity":"high","area":"stitcher / config wiring","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py:201,324","evidence":"Both VideoCompiler constructions in main.py are VideoCompiler(output_dir) with no padding args; compiler.py:10 defaults end_padding=1.0, begin_padding=0.5. config.json:63-64 and StitchingConfig (models.py:86-87) say 0.5/0.5. Grep confirms the only caller honoring config padding is the interactive demo run_process_and_stitch.py:134. StitchingConfig.use_cache (models.py:88) is read nowhere; min_shot_count is used only by the demo script. CODE_MAP's ramp table ('Change stitching padding | config.json stitching.{begin_padding,end_padding}') is wrong for the API/CLI path. gemini.py:380-381 reads the config values but only to LOG an estimated reel duration ? so the logged estimate disagrees with the actual reel.","why_it_matters":"A documented, shipped config knob does nothing on the primary /api/compile and --trim paths, and the effective end padding (1.0s) silently differs from the configured value (0.5s) ? every compiled highlight is 0.5s/clip longer than configured, and tuning padding via config has no effect. NEW finding."}
Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:12:55.882Z)

```
run_process_and_stitch.py:134:    compiler = VideoCompiler(output_dir, end_padding=end_padding,
begin_padding=begin_padding)
backend\main.py:201:    compiler = VideoCompiler(output_dir)
backend\main.py:324:        compiler = VideoCompiler(out_dir)
```

3. user (2026-06-10T06:12:56.879Z)

```
config.json:43:        "ai_handoff_padding": 2.0,
config.json:63:        "begin_padding": 0.5,
config.json:64:        "end_padding": 0.5,
README.md:20:[Omitted long matching line]
README.md:28:*   **High-Speed FFmpeg Stitching** with configurable `begin_padding`,
`end_padding`, and `min_shot_count` filtering.
README.md:99:`run_process_and_stitch.py` is an interactive convenience runner: it MD5-hashes a
hardcoded video path, reuses cached indexed segments if available (offering to re-index from
scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`,
`min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching
settings against a fixed large recording without re-running expensive segmentation.
README.md:137:        "ai_handoff_padding": 2.0,
README.md:145:        "begin_padding": 0.5,
README.md:146:        "end_padding": 0.5,
README.md:160:*   `indexing.ai_handoff_padding` ? seconds of slack added around each YOLO
candidate window before sending to the AI provider.
README.md:226:3. **AI handoff** ? only the surviving candidate windows (padded by
`ai_handoff_padding` seconds on each side) are extracted and sent to the AI provider for precise
rally boundaries and shot-count estimation. These calls run in parallel up to `ai_max_workers`.
run_process_and_stitch.py:87:    end_padding = cfg.stitching.end_padding
run_process_and_stitch.py:88:    begin_padding = cfg.stitching.begin_padding
run_process_and_stitch.py:94:            bpad_input = input(f"Enter begin_padding in seconds
(default {begin_padding}): ").strip()
run_process_and_stitch.py:96:                begin_padding = float(bpad_input)
run_process_and_stitch.py:98:            pad_input = input(f"Enter end_padding in seconds
(default {end_padding}): ").strip()
run_process_and_stitch.py:100:                end_padding = float(pad_input)
run_process_and_stitch.py:108:    print(f"\nCompiling highlights (with
begin_padding={begin_padding}s, end_padding={end_padding}s,
min_shot_count={min_shot_count})...", flush=True)
run_process_and_stitch.py:134:    compiler = VideoCompiler(output_dir, end_padding=end_padding,
begin_padding=begin_padding)
docs\CODE_MAP.md:34:  -> select segment ids -> [(start,end)] (+ stitching padding)
docs\CODE_MAP.md:42: -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt)   ? provider_registry
docs\CODE_MAP.md:87:- `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths,
interactive `input()` (blocks automation), skips validation. Config via
`backend.config.load_config` (typed); segmenter, padding, and db path all read from the model
(no literal fallbacks).
docs\CODE_MAP.md:96:[Omitted long matching line]
docs\CODE_MAP.md:97:[Omitted long matching line]
docs\CODE_MAP.md:110:[Omitted long matching line]
docs\CODE_MAP.md:149:[Omitted long matching line]
docs\CODE_MAP.md:150:- `@backend/tools/rally_slicer.py` ? standalone CLI.
`::compute_clip_windows` (pure, unit-tested), `::load_rally_labels` (golden schema; ? silently
skips degenerate rows), `::slice_rallies`, `::ClipWindow`(rally:str),
`::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (? duplicate compiler's by copy). `_read_time ->
backend/eval/annotations.parse_timestamp`.
docs\CODE_MAP.md:254:- **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`,
slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
docs\CODE_MAP.md:268:| Change stitching padding | `config.json
stitching.{begin_padding,end_padding}`; impl
`@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
docs\EVALUATION.md:138:| Recall looks low only at high `--iou` | Rallies *are* found but
boundaries are loose ? check `dStart`/`dEnd`. Tune `stitching` padding or the segmentation
```

```
thresholds rather than the detector. |
backend\api\static\style.css:36:    padding: 0;
backend\api\static\style.css:68:    padding: 40px 24px;
backend\api\static\style.css:80:    padding-bottom: 24px;
backend\api\static\style.css:101:    padding: 8px 16px;
backend\api\static\style.css:138:    padding: 24px;
backend\api\static\style.css:163:    padding-bottom: 12px;
backend\api\static\style.css:183:    padding: 12px 16px;
backend\api\static\style.css:201:    padding: 14px 20px;
backend\api\static\style.css:226:    padding: 12px 20px;
backend\api\static\style.css:254:    padding: 12px 16px;
backend\api\static\style.css:314:    padding: 4px 8px;
backend\api\static\style.css:347:    padding: 16px;
backend\api\static\style.css:356:    padding: 8px 12px;
backend\api\static\style.css:374:    padding-right: 6px;
backend\api\static\style.css:381:    padding: 16px 20px;
backend\api\static\style.css:445:    padding: 2px 8px;
backend\api\static\style.css:511:    padding: 20px 24px;
backend\api\static\style.css:585:    padding: 60px 40px;
backend\api\static\index.html:54:                        <div class="empty-state"
style="padding: 20px;">
backend\api\static\index.html:66:                        <h2 class="panel-title" style="border:
none; padding: 0;">
backend\api\static\index.html:100:                         <button id="btn-run-query" class="btn-
secondary" style="height: 38px; margin-top: auto; padding: 0;">
backend\api\static\index.html:124:                             <input type="text" id="highlight-
output-name" class="text-input" style="padding: 8px 12px; font-size: 0.85rem; width: 220px;"
value="highlights_compiled.mp4">
backend\config\models.py:64:    ai_handoff_padding: float = 2.0
backend\config\models.py:86:    begin_padding: float = 0.5
backend\config\models.py:87:    end_padding: float = 0.5
tests\test_config.py:131:    assert cfg.get("begin_padding") == 0.5          # lives inside
`stitching`
backend\tools\rally_slicer.py:13:Padding matches the highlight compiler (begin 0.5 s / end 1.0
s) so a sliced clip
backend\tools\rally_slicer.py:30:BEGIN_PADDING = 0.5
backend\tools\rally_slicer.py:31:END_PADDING = 1.0
backend\tools\rally_slicer.py:78:                         begin_pad: float = BEGIN_PADDING,
backend\tools\rally_slicer.py:79:                         end_pad: float = END_PADDING,
backend\tools\rally_slicer.py:81:    """Apply padding and clamp to the video, for the requested
rallies.
backend\tools\rally_slicer.py:119:                 out_dir: str, begin_pad: float =
BEGIN_PADDING,
backend\tools\rally_slicer.py:120:                 end_pad: float = END_PADDING) -> List[str]:
backend\tools\rally_slicer.py:165:    p.add_argument("--begin-pad", type=float,
default=BEGIN_PADDING)
backend\tools\rally_slicer.py:166:    p.add_argument("--end-pad", type=float,
default=END_PADDING)
tests\test_rally_slicer.py:5:    load_rally_labels, compute_clip_windows, ClipWindow,
BEGIN_PADDING, END_PADDING,
tests\test_rally_slicer.py:46:def test_padding_applied():
tests\test_rally_slicer.py:48:    assert w == [ClipWindow("1", 10.0 - BEGIN_PADDING, 20.0 +
END_PADDING)]
backend\pipeline\segmenters\gemini.py:380:        begin_pad = stitch_cfg.get("begin_padding",
0.5)
backend\pipeline\segmenters\gemini.py:381:        end_pad = stitch_cfg.get("end_padding", 0.5)
backend\pipeline\segmenters\gemini.py:391:            f"({total_highlight_dur:.1f}s total;
includes {begin_pad}s begin + {end_pad}s end padding per segment)"
backend\pipeline\stitcher\compiler.py:10:    def __init__(self, output_dir: str, end_padding:
float = 1.0, begin_padding: float = 0.5):
backend\pipeline\stitcher\compiler.py:12:        self.end_padding = end_padding
backend\pipeline\stitcher\compiler.py:13:        self.begin_padding = begin_padding
backend\pipeline\stitcher\compiler.py:67:        into a seamless highlights file, applying
end_padding to prevent abrupt endings.
backend\pipeline\stitcher\compiler.py:90:            logger.info(f"Trimming {len(segments)}
```

```
segments (begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
backend\pipeline\stitcher\compiler.py:125:                padded_start = max(0.0, start -
self.begin_padding)
backend\pipeline\stitcher\compiler.py:126:                padded_end = end + self.end_padding
backend\pipeline\stitcher\compiler.py:131:                    logger.info(f"{segment_label}:
Padded end ({end + self.end_padding:.2f}s) exceeds video duration. "
backend\pipeline\stitcher\compiler.py:132:                                f"Clamping to
{video_duration:.2f}s (end padding partially applied).")
backend\pipeline\segmenters\tracknet_hybrid.py:102:        handoff_padding =
idx_cfg.get("ai_handoff_padding", 2.0)
backend\pipeline\segmenters\tracknet_hybrid.py:181:            w_start = max(0, window["start"]
- handoff_padding)
backend\pipeline\segmenters\tracknet_hybrid.py:182:            w_end = min(video_duration,
window["end"] + handoff_padding)
backend\pipeline\segmenters\wasb_hybrid.py:93:        handoff_padding =
idx_cfg.get("ai_handoff_padding", 2.0)
backend\pipeline\segmenters\wasb_hybrid.py:161:            w_start = max(0, window["start"] -
handoff_padding)
backend\pipeline\segmenters\wasb_hybrid.py:162:            w_end = min(video_duration,
window["end"] + handoff_padding)
backend\pipeline\segmenters\yolo_hybrid.py:309:        handoff_padding =
idx_cfg.get("ai_handoff_padding", idx_cfg.get("gemini_handoff_padding", 2.0))
backend\pipeline\segmenters\yolo_hybrid.py:486:            w_start = max(0, window["start"] -
handoff_padding)
backend\pipeline\segmenters\yolo_hybrid.py:487:            w_end = min(video_duration,
window["end"] + handoff_padding)
```

4. user (2026-06-10T06:13:12.133Z)

```
170                 response["reports"] = paths
171
172     @app.post("/api/query")
173     def query_play_segments(req: QueryRequest):
174         filters = {
175             "server": req.server,
176             "receiver": req.receiver,
177             "duration_min": req.duration_min,
178             "event_type": req.event_type
179         }
180         segments = db_instance.query_play_segments(req.video_id, filters)
181         return {"play_segments": segments}
182
183     @app.post("/api/compile")
184     def compile_highlights(req: CompileRequest):
185         video = db_instance.get_video(req.video_id)
186         if not video:
187             raise HTTPException(status_code=404, detail="Indexed video record not found.")
188
189         # Retrieve matching play segments from db
190         all_segments = db_instance.query_play_segments(req.video_id, {})
191         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
192
193         if not matched_segments:
194             raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
195
196         # Extract start/end time pairs
197         time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]
198
199         # Run compiler
200         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
201         compiler = VideoCompiler(output_dir)
202         try:
203             out_path = compiler.compile_highlights(video["filepath"], time_pairs,
```

```
req.output_filename)
204             return {"status": "success", "filepath": out_path}
205         except Exception as e:
206             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}")
207
208     # --- CLI Debug Utility & Orchestration ---
209     def run_cli_diagnose_clip(video_path: str, timestamp: float):
210         """CLI debugging utility to examine segment properties around a timestamp."""
211         print("=" * 60)
212         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
213         print("=" * 60)
214
215         cfg = load_config()  # legacy wrapper still works, returns dict for now
216
217         # 1. Validation check diagnostics
218         print("[DIAGNOSTIC] Running validation framework...")
219         results = registry.run_all(video_path, cfg)
220         for r in results:
221             status_symbol = "[PASS]" if r.passed else "[FAIL]"
222             print(f"  {status_symbol} {r.validator_name}: {r.message}")
223             if r.details:
224                 print(f"         Details: {r.details}")
225
226         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
227         print("-" * 60)
228         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
229         import cv2
```

5. user (2026-06-10T06:13:13.146Z)

```
295
296         args = parser.parse_args()
297
298         if args.setup:
299             # Invoke setup diagnostics
300             import subprocess
301             print("[INFO] Invoking setup.py...")
302             subprocess.run([sys.executable, "setup.py"])
303             return
304
305         if args.trim_start is not None and args.trim_end is not None:
306             if not args.video_path:
307                 print("[ERROR] Please provide --video-path to extract a segment.")
308                 return
309
310             # Determine output path
311             out_filename = args.trim_output or "trimmed_segment.mp4"
312             if os.path.isabs(out_filename):
313                 out_path = out_filename
314                 out_dir = os.path.dirname(out_path)
315                 out_name = os.path.basename(out_path)
316             else:
317                 out_dir = args.output_dir
318                 out_path = os.path.join(out_dir, out_filename)
319                 out_name = out_filename
320
321             os.makedirs(out_dir, exist_ok=True)
322             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
323
324             compiler = VideoCompiler(out_dir)
325             try:
326                 res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
```

```
                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig

        # The CLI --output-dir overrides the configured output directory. It now lives
        # on the typed model (OutputConfig.output_dir), so every consumer ? including
```

6. user (2026-06-10T06:13:13.550Z)

```
import os
import subprocess
import tempfile
import logging
from typing import List, Tuple, Union

logger = logging.getLogger(__name__)

class VideoCompiler:
    def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5):
        self.output_dir = output_dir
        self.end_padding = end_padding
        self.begin_padding = begin_padding
        os.makedirs(output_dir, exist_ok=True)

    @staticmethod
    def _parse_time(val: Union[int, float, str]) -> float:
        """Normalizes a timestamp value to float seconds.
        Handles: float/int (pass-through), plain numeric strings ("12.4"),
        and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
        This mirrors the parse_time logic used in gemini.py to ensure
        the stitcher can always consume whatever format the indexer produced."""
        if isinstance(val, (int, float)):
            return float(val)
        if isinstance(val, str):
            val = val.strip()
            if ":" in val:
                # Handle MM:SS, HH:MM:SS, etc.
                parts = val.split(":")
                parts.reverse()
                total = 0.0
                for i, p in enumerate(parts):
                    try:
                        total += float(p) * (60 ** i)
                    except ValueError:
                        pass
                return total
            try:
                return float(val)
            except ValueError:
```

7. user (2026-06-10T06:13:29.079Z)

88   - `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed.
`::_default_db_path` resolves via `backend.config.load_config`
(`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
89   - `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label);
config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig`
(same object the live pipeline feeds segmenters).
90   - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
0=ok, 1=REGRESSION (CI gate), 2=missing DB. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run); `::_default_db_path` resolves via
`backend.config.load_config`.
91
92   - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports
`load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`,
`ValidationConfig` (from `.models`); `__all__` = exactly those 6. Use `from backend.config
import load_config, AppConfig`.
93   - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for
defaults**.
94   - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`,`indexing`,`output`,`stitching`
(`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
95   - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain
dicts** (`model_dump(mode='json')`, NOT the model) so chained
`config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any
section (sections discovered dynamically from `model_fields`). ? "bit every validator on real
runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises
`KeyError` for unknown top-level keys else delegates to `.get`.
96   - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`,
`min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`,
`chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,
`ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested
`tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `'gemini'`. ?
`::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ?
a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`.
97   - `::ValidationConfig` (`min_resolution_width=1280`,`min_resolution_height=720`,`min_f
ps=29.9`,`min_blur_threshold=20.0`,`max_scene_cuts_per_minute=0.5`, nested `relevance`);
`::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`).
`::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`,
`conda_env='TrackNetV4'`, `weights_path=''`, `queue_length=5`, `velocity_threshold=0.02`).
`::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`,
`output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`).
`::StitchingConfig`
(`begin_padding=0.5`,`end_padding=0.5`,`use_cache=False`,`min_shot_count=1`). `::Sport` Literal
of 5 sports. `::Config = AppConfig` alias.
98   - `@backend/config/loader.py` ? load/save with defaults-merge.
99   - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root).
`::_get_builtin_defaults` = `AppConfig().model_dump(...)`.
100   - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole
defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
101   - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present
(used by setup.py/`--setup`). `::get_default_config` ? ? transitional plain-dict alias.
102   - `@config.json` ? on-disk config; mirrors `AppConfig`. ?
`indexing.default_segmenter='gemini'` diverges from model default `'yolo_hybrid'` ? file
wins at runtime.
103   - `@setup.py` ? `::init_default_config` ? delegates to
`backend.config.save_default_config` (single-source defaults; skips if config.json exists).
`::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8,
ffmpeg/ffprobe on PATH, optional torch+CUDA).
104
105   ### 2.1 Segmenters (under pipeline/segmenters) ?

106   - `@backend/pipeline/segmenters/base.py`
107   - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract `name`,`description`,`process_video`. § `process_video(video_id, video_path, player_pool=None, max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.
108   - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` -> name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-instantiates + swallows exceptions into "No description available." (a throwing `description` is masked).
109   - `@backend/pipeline/segmenters/__init__.py` ? imports all 5 (`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest; re-exports them + `segmenter_registry` via `__all__`.
110   - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name `'wasb_hybrid'`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config `indexing.wasb.velocity_threshold(0.02)` + shared `dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs `{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit (copy-paste drift).
111   - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name `'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig` from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`; `detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably yolo_hybrid).
112   - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name `'yolo_hybrid'`, no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005). `::_PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections). `::_lazy_load_model` (lazy `torchvision` import so module/tests load without it), `::_extract_action_windows_from_chunk`, `::_make_tracker` (per-chunk ByteTrack ? track IDs chunk-local), `::_DETECTOR_FACTORIES`/`::_DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ? velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to `frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU); `yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated aliases still honored.
113   - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ? pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded; `max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning table).
114   - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM** (random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing server/receiver; uses `print()`; honors `max_frames`.
115
116   ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
117   - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() -> Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL de-risk seam (low-regret 6 / review item 1).

8. user (2026-06-10T06:13:30.032Z)

245   - **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs (`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
246   - **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg + `--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo modules).
247   - **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-

reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
248   - **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST exist in `output/<db_name>` before any eval/regression.
249   - **Validator ordering coupling:** exec order is Format -> ResolutionFPS -> PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless `format_check` ran earlier (shared `context['ffprobe_meta']`). PTS 10.0s gap + scene-cut 0.6 correlation are NOT config-overridable.
250   - **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-deletes its segments. Use `clear_video_data` deliberately. `_get_connection` MUST re-issue `PRAGMA foreign_keys=ON` per connection.
251   - **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs tracknet's `is None` check.
252   - **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs `backend.annotations` (provider registry). regression.py imports the latter.
253   - **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-CSV schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by `distill_local`; `harness.get_predictions` reused by `regression.regress`.
254   - **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`, slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
255   - **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
256   - **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat; if probe fails (0.0) no boundary validation.
257   - **Audio NOT in the live pipeline:** `pipeline/audio/hit_detector` + `eval/audio/e1_probe` are a diagnostic probe (ADR-007); recall 100% but discrimination ~2.2× -> not adopted for fusion.
258
259   ---
260
261   ## 7. RAMP PATHS (to do X -> open Y::symbol)
262 | Task | Open |
263 |---|---|
264 | Change a config default / add a config field | `@backend/config/models.py` (single source of truth) -> regenerate via `setup.py`/`save_default_config`; ? config.json may override (default_segmenter) |
265 | Read config in new code | `from backend.config import load_config, AppConfig` (`@backend/config/__init__.py`); attribute access, NOT raw `json.load` |
266 | Tune candidate windowing (velocity/merge/min-dur) | `@backend/pipeline/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`, `min_action_window_duration` |
267 | Add a segmenter | subclass `@backend/pipeline/segmenters/base.py::BasePlaySegmenter`; `segmenter_registry.register(...)` at module bottom; import in `@backend/pipeline/segmenters/__init__.py`; if AI-based add to `@backend/reporting/harvest.py::_AI_SEGMENTERS` |
268 | Change stitching padding | `config.json stitching.{begin_padding,end_padding}`; impl `@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror `@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
269 | Add a sport | subclass `@backend/sports/base.py::SportProfile`; `sport_registry.register(...)` in `@backend/sports/__init__.py`; define prompt+HSV in new file |
270 | Add an AI provider | subclass `@backend/providers/base.py::AIVideoProvider`; `provider_registry.register('name', Cls)` |
271 | Add an annotation/GT tier | subclass `@backend/annotations/base.py::AnnotationProvider`; register + import in `@backend/annotations/__init__.py` |
272 | Add / change a report footgun rule | `@backend/reporting/spec.yaml::rules` (declarative; ? `when.path` must match harvest's stage/metric/detail names); facts in `@backend/reporting/harvest.py` |
273 | Tweak what the report records | `@backend/reporting/harvest.py::record_run` + stage builders; emit point is `@backend/main.py` `finally:` -> `@backend/reporting/__init__.py::emit_indexer_report` |
274 | Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main` (`--trajectory --labels --fps --frame-width`); trust `cv` recall not `fit` |

9. user (2026-06-10T06:13:30.972Z)

```
78          default_highlights_name: str = "highlights.mp4"
79          db_name: str = "sports_indexer.db"
80          # Directory where the DB, highlight reels, and processed clips are written.
81          # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
82          output_dir: str = "output"
83
84
85      class StitchingConfig(BaseModel):
86          begin_padding: float = 0.5
87          end_padding: float = 0.5
88          use_cache: bool = False
89          min_shot_count: int = 1
90
91
92      class AppConfig(BaseModel):
93          """Root configuration object.
94
95          All runtime configuration should be accessed via an instance of this class
96          (or the loader) rather than raw dict lookups.
97          """
```

10. user (2026-06-10T06:13:31.629Z)

```
370                         non_overlapping.append(curr)
371                 validated = non_overlapping
372
373             # Log validation summary
374             if rejected:
375                 reject_lines = [f"  ? {label}: {reason}" for label, reason in rejected]
376                 logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
377             logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")
378
379             stitch_cfg = self.config.get("stitching", {})
380             begin_pad = stitch_cfg.get("begin_padding", 0.5)
381             end_pad = stitch_cfg.get("end_padding", 0.5)
382
383             total_highlight_dur = 0.0
384             for c in validated:
385                 total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
386
387             mins = int(total_highlight_dur // 60)
388             secs = int(total_highlight_dur % 60)
389             logger.info(
390                 f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
391                 f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
392             )
393
394             # 9. Populate SQLite DB with validated play segments
395             for c in validated:
396                 start = c["start"]
397                 end = c["end"]
```

11. user (2026-06-10T06:13:48.219Z)

```
config.json:65:        "use_cache": false,
config.json:66:        "min_shot_count": 1
README.md:28:*   **High-Speed FFmpeg Stitching** with configurable `begin_padding`,
`end_padding`, and `min_shot_count` filtering.
README.md:99:`run_process_and_stitch.py` is an interactive convenience runner: it MD5-hashes a
hardcoded video path, reuses cached indexed segments if available (offering to re-index from
```

```
scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`,
`min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching
settings against a fixed large recording without re-running expensive segmentation.
README.md:147:        "use_cache": false,
README.md:148:        "min_shot_count": 1
README.md:162:*   `stitching.min_shot_count` ? drop play segments whose estimated shot count is
below this threshold when compiling highlights.
run_process_and_stitch.py:89:    min_shot_count = cfg.stitching.min_shot_count
run_process_and_stitch.py:102:            shot_input = input(f"Enter min_shot_count to filter by
(default {min_shot_count}): ").strip()
run_process_and_stitch.py:104:                min_shot_count = int(shot_input)
run_process_and_stitch.py:108:    print(f"\nCompiling highlights (with
begin_padding={begin_padding}s, end_padding={end_padding}s,
min_shot_count={min_shot_count})...", flush=True)
run_process_and_stitch.py:110:    # Filter segments where shot_count >= min_shot_count
run_process_and_stitch.py:121:        if shot_count >= min_shot_count:
run_process_and_stitch.py:127:        print(f"[FILTER] Excluded {skipped_count} play segments
with fewer than {min_shot_count} shots.", flush=True)
docs\CODE_MAP.md:97:[Omitted long matching line]
backend\config\models.py:88:    use_cache: bool = False
backend\config\models.py:89:    min_shot_count: int = 1
```

12. user (2026-06-10T06:13:49.548Z)

```
No matches found
```

13. user (2026-06-10T06:14:22.160Z)

```
Structured output provided successfully
```

14. assistant (2026-06-10T06:14:28.950Z)

```
Verification complete. The finding is real: the config padding knob is dead on the live
`/api/compile` and `--trim` paths, with the docs actively pointing users to it.
```